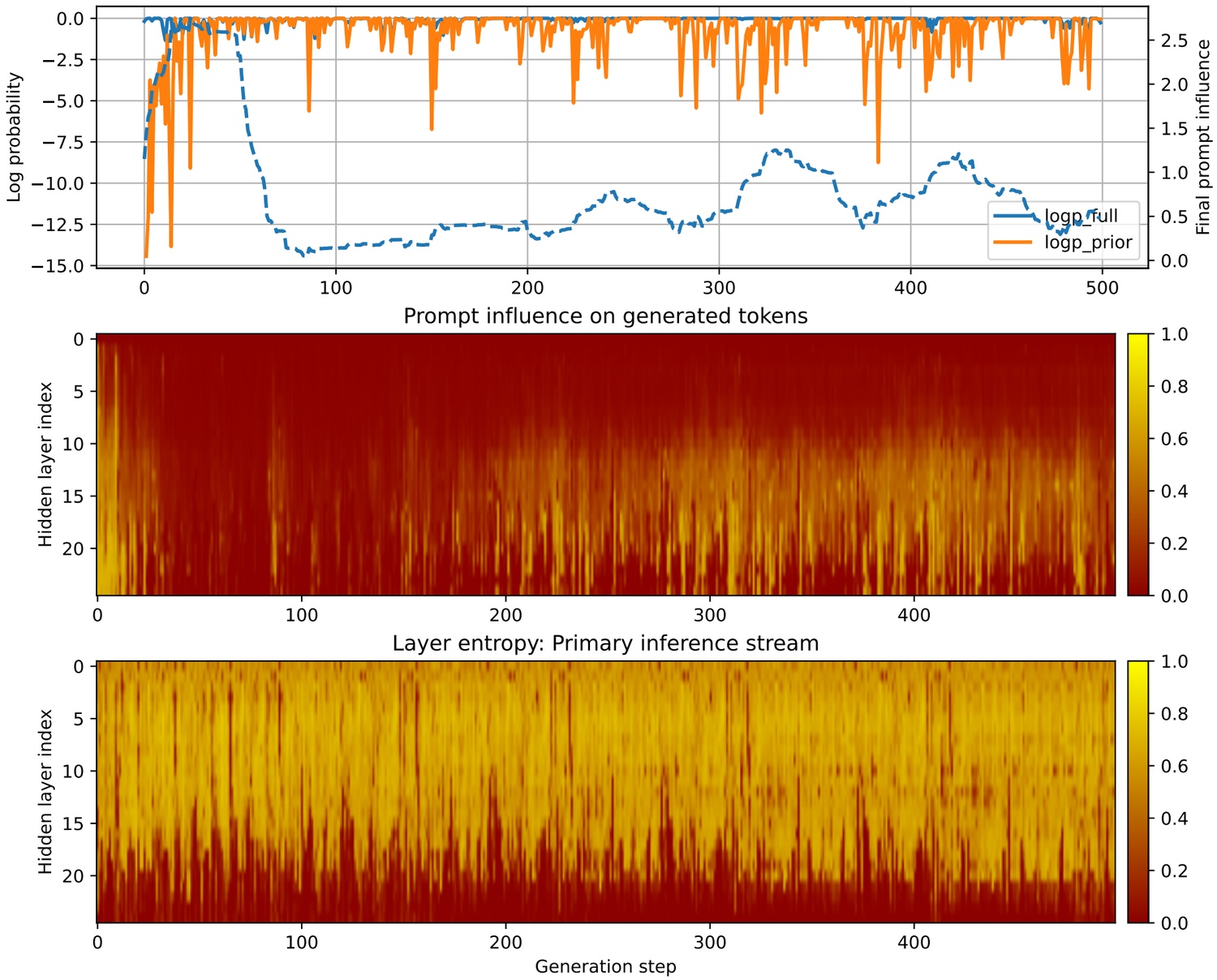
Prompt 1




```
PROMPT:
Task ID: PYC082  Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests.  Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC082  Your code must contain this required symbol:
def normalize_records  Implement this task:
write function normalize_records(rows,schema) filling missing
fields with defaults..  Start your answer with
the check-token line, then provide the Python implementation.

MODEL RESPONSE:
# CHECK_TOKEN: PYC082  # Function to normalize records based on a
given schema def normalize_records(rows,
schema):     # Check if the number of rows is valid     if
len(rows) < 1:         raise ValueError("Invalid
number of rows")      # Check if the schema is valid     if not
isinstance(schema, dict):         raise
TypeError("Schema must be a dictionary")      # Normalize the
schema     normalized_schema = {}     for key,
value in schema.items():         if key not in normalized_schema:
            normalized_schema[key] = value
else:             if not isinstance(value, (dict, list)):
        raise TypeError(f"Value for {key}
must be a dictionary or list")              # Normalize the value
            normalized_value = {}
for sub_key, sub_value in value.items():                 if
sub_key not in normalized_value:
normalized_value[sub_key] = sub_value                 else:
            if not isinstance(sub_value,
(dict, list)):                         raise TypeError(f"Value
for {sub_key} must be a dictionary or list")
# Normalize the sub-value
normalized_sub_value = {}                             for sub_sub_key,
sub_sub_value in sub_value.items():                             if
sub_sub_key not in normalized_sub_value:
normalized_sub_value[sub_sub_key] = sub_sub_value
        else:                             if
not isinstance(sub_sub_value, (dict, list)):
          raise TypeError(f"Value for
{sub_sub_key} must be a dictionary or list")
        # Normalize the sub-sub-value
normalized_sub_sub_value = {}                                 for
sub_sub_sub_key, sub_sub_sub_value in
sub_sub_value.items():                                 if
sub_sub_sub_key not in normalized_sub_sub_value:
normalized_sub_sub_value[sub_sub_sub_key] = sub_sub_sub_value
                            else:
if not isinstance(sub_sub_sub_value, (dict, list)):
                        raise
TypeError(f"Value for {sub_sub_sub_key} must be a dictionary or
list")                                 #
Normalize the sub-sub-sub-value
  normalized_sub_sub_sub_value = {}
for sub_sub_sub_sub_key, sub_sub_sub_sub_value in
sub_sub_sub_value.items():
if sub_sub_sub_sub_key not in normalized_sub_sub_sub_value:
normalized_sub_sub_sub_value[sub_sub_sub_sub_key] =
sub_sub_sub_sub_value
normalized_value[sub_key] =
normalized_sub_value[sub_key].update(normalized_sub_sub_value[sub
```